Chapter 10

The algorithm toolkit

the right container turns a slow program into a fast one

Lists, dicts and sets solve most problems. A handful of standard-library modules cover almost everything else that algorithm courses and interviews ask for. Each one exists because a common operation is **slow** with the obvious structure:

you need to…	obvious way	cost	use instead	cost
count things	dict + `if k in d`	fiddly	`collections.Counter`	O(n)
take from the front of a queue	`list.pop(0)`	**O(n)**	`deque.popleft()`	O(1)
repeatedly get the smallest	`min(xs)` then remove	**O(n)**	`heapq.heappop`	O(log n)
find a position in sorted data	scan the list	**O(n)**	`bisect.bisect_left`	O(log n)
test membership many times	`x in some_list`	**O(n)**	`x in some_set`	O(1)
every pair / combination	nested loops	fiddly	`itertools.combinations`	lazy

The difference is not academic. The same job, 50,000 items, measured on this machine while building the book:

```python
import time
from collections import deque

def drain_list(n):
    q = list(range(n))
    while q:
        q.pop(0)                # shifts every remaining item left

def drain_deque(n):
    q = deque(range(n))
    while q:
        q.popleft()             # just moves a pointer

for fn in (drain_list, drain_deque):
    t = time.perf_counter(); fn(50_000); ms = (time.perf_counter() - t) * 1000
    print(f"{fn.__name__:12} {ms:8.1f} ms")
```

```
drain_list      118.0 ms
drain_deque       1.7 ms
```

KEY IDEA

Big-O (Ch 20) tells you how the cost grows. O(n) per operation inside a loop of n is O(n²) overall: 50,000 items means ~1.25 billion element moves. Choosing the structure is the optimisation; micro-tuning the loop rarely matters.

collections: Counter, defaultdict, deque

```python
from collections import Counter, defaultdict

words = "the cat and the hat and the bat".split()
c = Counter(words)
print(c.most_common(2), c["the"], c["dog"])          # missing keys count as 0
print(c + Counter(["cat", "dog"]))                     # counters add like multisets

by_letter = defaultdict(list)                          # missing key -> list() made for you
for w in words:
    by_letter[w[0]].append(w)
print(dict(by_letter))
```

```
[('the', 3), ('and', 2)] 3 0
Counter({'the': 3, 'cat': 2, 'and': 2, 'hat': 1, 'bat': 1, 'dog': 1})
{'t': ['the', 'the', 'the'], 'c': ['cat'], 'a': ['and', 'and'], 'h': ['hat'], 'b': ['bat']}
```

plain dict

```python
groups = {}
for w in "ab ac bd".split():
    if w[0] not in groups:
        groups[w[0]] = []
    groups[w[0]].append(w)
print(groups)
```

```
{'a': ['ab', 'ac'], 'b': ['bd']}
```

defaultdict

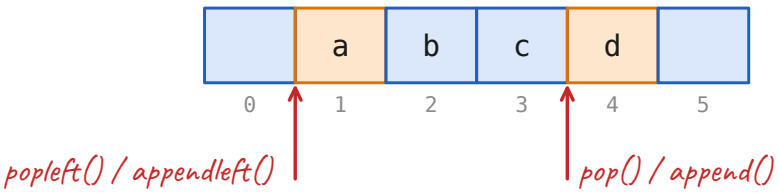
```python
from collections import defaultdict
groups = defaultdict(list)
for w in "ab ac bd".split():
    groups[w[0]].append(w)

print(dict(groups))
```

```
{'a': ['ab', 'ac'], 'b': ['bd']}
```



deque: a queue with two fast ends



```python
from collections import deque

recent = deque(maxlen=3)              # keeps only the last 3: older items fall off the left
for page in ["home", "dune", "emma", "ubik", "kim"]:
    recent.append(page)
print(recent, recent[0])

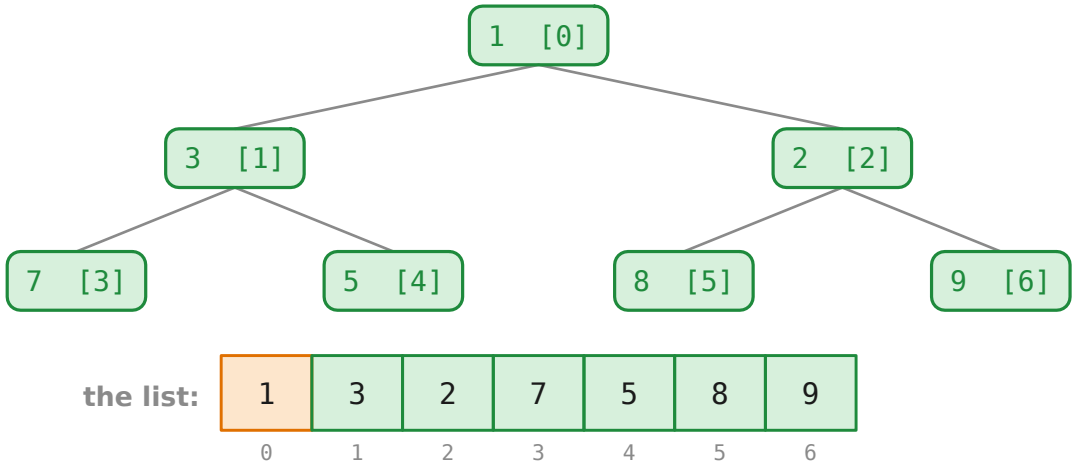
q = deque(["ana", "ben"])
q.append("cy"); first = q.popleft(); q.rotate(1)
print(first, q)
```

```
deque(['emma', 'ubik', 'kim'], maxlen=3) emma
ana deque(['cy', 'ben'])
```

also in collections	for
`OrderedDict`	`move_to_end()`, `popitem(last=False)`: an LRU cache in a few lines (practice 10.6)
`ChainMap(a, b)`	look up in `a`, then `b`: layered config (command line > env > defaults)
`namedtuple("P", "x y")`	a tuple with field names (prefer `typing.NamedTuple` or a dataclass)

heapq: always know the smallest

A **heap** is a list kept in a special order: every parent is ≤ its children, so the smallest item is always at index 0. It's a binary tree stored flat in a list: the children of index `i` are at `2i+1` and `2i+2`. Push and pop are O(log n) because only one root-to-leaf path is fixed up:




```python
import heapq

h = [7, 3, 9, 1, 5, 8, 2]
heapq.heapify(h)                          # O(n), in place
print(h, h[0])
heapq.heappush(h, 0)
print([heapq.heappop(h) for _ in range(3)])

tasks = []                                # (priority, order, item): ties broken by order
for i, (pri, job) in enumerate([(2, "email"), (1, "backup"), (2, "report")]):
    heapq.heappush(tasks, (pri, i, job))
print([heapq.heappop(tasks)[2] for _ in range(len(tasks))])

scores = {"ana": 81, "ben": 67, "cy": 94, "di": 72}
print(heapq.nlargest(2, scores, key=scores.get))     # top k without a full sort
```

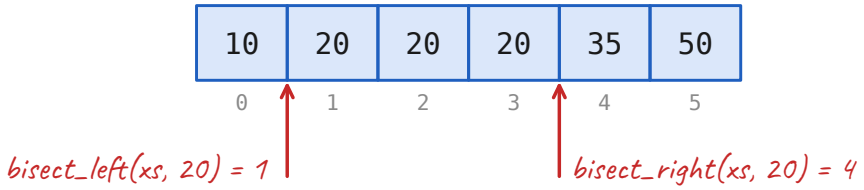
```
[1, 3, 2, 7, 5, 8, 9] 1
[0, 1, 2]
['backup', 'email', 'report']
['cy', 'ana']
```

TRAP

`heapq` is a **min**-heap only. For a max-heap push negated keys (`-score`). And a heap list is not sorted: only `h[0]` is guaranteed to be the minimum.

bisect: binary search in sorted data

`bisect` finds where a value belongs in a sorted list by halving the search range each step: about 17 steps for 100,000 items instead of up to 100,000:




```python
import bisect

xs = [10, 20, 20, 20, 35, 50]
print(bisect.bisect_left(xs, 20), bisect.bisect_right(xs, 20))
print(bisect.bisect_right(xs, 20) - bisect.bisect_left(xs, 20), "copies of 20")

bisect.insort(xs, 27)                       # insert keeping it sorted
print(xs)

def grade(score, cutoffs=(40, 50, 60, 70), letters="FDCBA"):
    return letters[bisect.bisect_right(cutoffs, score)]
print([grade(s) for s in (12, 40, 55, 69, 70, 99)])
```

```
1 4
3 copies of 20
[10, 20, 20, 20, 27, 35, 50]
['F', 'D', 'C', 'B', 'A', 'A']
```

WHY

`bisect_left` gives the first position where the value could go (before equal items), `bisect_right` the last (after them). Their difference counts the equal items; `bisect_right` on cutoffs turns a chain of `if score >= 70 ... elif` into one line.

itertools: loops you don't have to write

```python
from itertools import (accumulate, chain, combinations, count, groupby, islice,
                       pairwise, permutations, product, batched)

print(list(combinations("ABC", 2)), len(list(permutations("ABC"))))
print(list(product("xy", [0, 1])))                      # nested loops, flattened
print(list(accumulate([3, 1, 4, 1, 5])))                # running totals
print(list(pairwise([1, 4, 9, 16])))                    # neighbours
print(list(batched("ABCDEFG", 3)))                      # chunks (3.12+)
print(list(islice(count(10, 5), 4)), list(chain("ab", [1, 2])))
```

```
[('A', 'B'), ('A', 'C'), ('B', 'C')] 6
[('x', 0), ('x', 1), ('y', 0), ('y', 1)]
[3, 4, 8, 9, 14]
[(1, 4), (4, 9), (9, 16)]
[('A', 'B', 'C'), ('D', 'E', 'F'), ('G',)]
[10, 15, 20, 25] ['a', 'b', 1, 2]
```

`groupby` groups **consecutive** equal keys, so it needs sorted input to group everything:

unsorted: surprise

```python
from itertools import groupby
ws = ["ant", "bee", "ape", "bat"]
print([(k, len(list(g)))
       for k, g in groupby(ws, key=lambda w: w[0])])
```

```
[('a', 1), ('b', 1), ('a', 1), ('b', 1)]
```

sorted first

```python
from itertools import groupby
ws = sorted(["ant", "bee", "ape", "bat"])
print([(k, len(list(g)))
       for k, g in groupby(ws, key=lambda w: w[0])])
```

```
[('a', 2), ('b', 2)]
```

function	gives	count for n items
`product(xs, repeat=k)`	every k-tuple, with repeats, order matters	n^k
`permutations(xs, k)`	ordered arrangements, no repeats	n! / (n−k)!
`combinations(xs, k)`	unordered selections, no repeats	n! / (k!(n−k)!)
`combinations_with_replacement`	unordered, repeats allowed	(n+k−1)! / (k!(n−1)!)

TRAP

These grow explosively: `permutations(range(12))` is 479 million tuples. They're lazy, so creating one is free, but `list()` on it will exhaust memory. Loop over them and stop early.

functools and operator

```python
from functools import reduce, cache, partial, cmp_to_key
from operator import mul, itemgetter

print(reduce(mul, range(1, 6)))                          # 1*2*3*4*5

@cache
def ways(n):                                             # staircase: 1 or 2 steps at a time
    return 1 if n <= 1 else ways(n - 1) + ways(n - 2)
print(ways(80))

def by_length_then_reverse_alpha(a, b):                  # old-style comparator: <0, 0, >0
    return (len(a) - len(b)) or ((a < b) - (a > b))
print(sorted(["bb", "a", "ccc", "aa"], key=cmp_to_key(by_length_then_reverse_alpha)))

rows = [("ana", 3), ("ben", 1), ("cy", 2)]
print(sorted(rows, key=itemgetter(1)), max(rows, key=itemgetter(1))[0])
```

```
120
37889062373143906
['a', 'bb', 'aa', 'ccc']
[('ben', 1), ('cy', 2), ('ana', 3)] ana
```

structure	add	remove	find min/max	x in s	notes
`list`	O(1) end	O(n) front	O(n)	O(n)	indexing O(1)
`deque`	O(1) both ends	O(1) both ends	O(n)	O(n)	indexing middle O(n)
`dict` / `set`	O(1)	O(1)	O(n)	**O(1)**	hashable keys; order kept (dict)
`heapq` list	O(log n)	O(log n) min	**O(1)** min	O(n)	min-heap only
sorted list + `bisect`	O(n) insort	O(n)	O(1) ends	O(log n)	great when read-mostly

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 10.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 10.1` shows other valid solutions and their trade-offs.

`10.1` build from a spec The k most frequent words
`10.2` build from a spec Merge sorted streams lazily
`10.3` measure and explain Pairs that sum to a target, fast
`10.4` build from a spec Running median with two heaps
`10.5` build from a spec Binary search questions
`10.6` mini-project Mini-project: an LRU cache

PASSPORT STAMP ask the same question of every new language (Appendix B)

6. Data structures

Beyond list/dict/set: `Counter`, `defaultdict`, `deque`, `OrderedDict`, `heapq` (min-heap on a list), `bisect` (binary search), lazy `itertools`, `functools.cache`/`reduce`.

12. Stdlib and tooling

Know each structure's costs; pick by the operation you do most.

IN OTHER LANGUAGES

Every language ships the same toolkit under other names: Java `ArrayDeque`, `PriorityQueue`, `Collections.binarySearch`, `HashMap`; C++ `std::deque`, `std::priority_queue` (a max-heap!), `std::lower_bound`; Rust `VecDeque`, `BinaryHeap`, `slice::binary_search`.

DEEP DIVE (your vault)

Vault notes: 7.2 A minimal cost vocabulary · 7.10 Choosing among sequence representations · 7.11 Sorting and ordering contracts · 7.15 A structure-selection case study · 11.11 collections · 11.12 itertools · 11.13 functools.